## Thông tin đề thi

**Course**: SDN302 – Software Design and Architecture using NodeJS **Semester**: FALL 2025, Block 3 **Tools**: Visual Studio Code (VS Code), MongoDB Compass, Postman or Thunder Client (offline)

### INSTRUCTIONS

Read the instructions below carefully before answering the questions:

- You may refer to the materials on your computer, your notebooks, and your textbooks.
- You are **NOT allowed** to share data with others by means of electronic devices or any other method.
- Avoid arbitrarily disconnecting from the **FPTU_EXAMONLINE** network while doing the test. If any network problems occur, immediately notify the exam supervisor for assistance.

**In addition to the conditions above, students must also meet the following requirements:**

1. Complete the exam using **Visual Studio Code** (VS Code), **MongoDB Compass**, **Postman** or **Thunder Client** (an extension for VSCode) tools (offline)
2. Using **NodeJS platform version 22.11.0**
3. Use the browser software available on your computer

**This part is very important, please read carefully and follow the instructions:**

- The data files of the test are provided in the **given** directory.
- You must use the solution provided in advance via email and install the additional library included in the accompanying request.
- Downloading any additional libraries while taking the test is **not allowed**
- Violating any of the above means your exam is considered invalid

**Instructions for submission:**

- Before submitting the solution, delete the **[node_modules]** folders in your test to reduce the space, to meet the allowed capacity of the **PEA_Client** software
- Use **PEA_Client** software to browse the **resource** directory to complete the submission process.

---

## Yêu cầu thiết lập

**Requirements:**

Using **MongoDB Compass**, create a database named "**SDN302_FA25_B5**". Create **collections** matching the data in the **given** directory, then import the data into those collections.

Database: **SDN302_FA25_B5** → Collections: **customers**, **orders**, **products**

Based on the given materials, build a **RESTFul API application** that fulfills the requirements of the problem:

**Context:**

This system manages customers, products, and orders for a basic e-commerce platform. Data is organized into three collections: **customers**, **products**, and **orders**. Students must build the required API endpoints using **ExpressJS, MongoDB, and Mongoose**. Each task specifies an API operation to implement, covering input validation, business rules, database processing, and proper JSON response formatting. Developers must use **Postman**, or **Thunder Client** to test their APIs.

**Note: 0 will be given to the work that:**

- Not using connection string to MongoDB in the **.env** file
- Not configuring the root path of RESTFul API Web App at: http://localhost:9999

---

## Question 1 (2.5 points): Retrieve Customer Purchase History

**Objective:**

Return all orders of a specified customer based on the customer ID, with brief product information included in each order.

*— Tài liệu được thành viên đăng tải tại **FUOVERFLOW** và được biên soạn lại để hỗ trợ ôn tập —*

**Endpoint:** `GET /api/customers/{customerId}/orders`

**Request Parameter:** customerId (ObjectId)

**Response 200 (OK):**

```
[
  {
    "_id": "6936f4c6f2937fc51e5f7cb1",
    "order_date": "2025-12-02T10:15:00.000Z",
    "status": "Pending",
    "items": [
```

```
      {
        "product": "Logitech Mouse",
        "quantity": 2,
        "total": 1000000
      },
      {
        "product": "Mechanical Keyboard",
        "quantity": 1,
        "total": 1500000
      }
    ],
    "total_amount": 2500000
  }
]
```

**Response 404 (Not Found):** If order not found

```
{
  "error": "No orders found for this customer"
}
```

---

## Question 2 (2.5 points): Product Sales Ranking Report

**Objective:**

Create a report showing the total quantity sold of each product, sorted by the highest sales.

**Endpoint:** `GET /api/orders/reports/product-sales`

**Response 200 (OK):**

```
[
  {
    "total_quantity_sold": 3,
    "product": "Mechanical Keyboard"
  },
  {
    "total_quantity_sold": 2,
    "product": "Logitech Mouse"
  },
  ……
]
```

**Business Rules:**

- Count total quantity sold for each product across all orders.
- Sort results by **total_quantity_sold** in descending order.
- Include only products that appear in at least one order.

---

## Question 3 (3 points): Create a New Order

**Objective:**

Place a new order, check stock availability, and automatically update product inventory.

**Endpoint:** `POST /api/orders`

**Request Body - Sample:**

```
{
  "customer_id": "6936f4c6f2937fc51e5f7ca8",
  "items": [
    { "product_id": "6936f4c6f2937fc51e5f7cab", "quantity": 2 },
    { "product_id": "6936f4c6f2937fc51e5f7cac", "quantity": 10 }
  ],
  "payment_method": "Credit Card"
}
```

**Response 201 (Created):**

```
{
  "message": "Order created successfully",
  "order_id": "693aa018083b21879dc5f3a2",
  "updated_products": [
    {
```

```
      "name": "Laptop Dell XPS 15",
      "stock": 18
    },
    {
      "name": "Logitech Mouse",
      "stock": 90
    }
  ],
  "total_amount": 75000000
}
```

**Response 404 (Not Found):** If customer not found

```
{
  "error": "Customer not found"
}
```

**Response 400 (Bad Request):** If items (product_id, quantity) not exist in request body

```
{
  "error": "Items required"
}
```

**Response 404 (Not Found):** If product_id in items not found in the products collection

```
{
  "error": "Product not found"
}
```

**Response 400 (Bad Request):** Insufficient stock

```
{
  "error": "Insufficient quantity to purchase"
}
```

**Business Rules:**

- Validate `customer_id`.
- For each product:
  - Reject request if `stock < quantity`.
  - Set `price_at_order = product.price`.
- After order creation, decrease stock accordingly.
- Recalculate `total_amount` on the server side.

---

## Question 4 (2 points): Update Order Items (Only If Status Is Pending)

**Objective:**

Update the items of an order only while its status is still "**Pending**".

**Endpoint:** `PUT /api/orders/{orderId}`

**Request Parameter:** `orderId` (ObjectId, required)

— Tài liệu được thành viên đăng tải tại ***FUOVERFLOW*** và được biên soạn lại để hỗ trợ ôn tập —

**Request Body:**

```
{
  "items": [
    {"product_id": "6936f4c6f2937fc51e5f7cae", "quantity": 10}
  ]
}
```

**Response 200 (OK):**

```
{
  "message": "Order updated successfully",
  "new_total_amount": 55000000
}
```

**Response 404 (Not Found):** If {orderId} not found in the orders collection

```
{
  "error": "Order not found"
}
```

**Response 400 (Bad Request):** If items (product_id, quantity) not exist in request body

```
{
  "error": "Items required"
}
```

**Response 409 (Conflict):** If status of the order not "Pending"

```
{
  "error": "Order is not in Pending status. Update denied."
}
```

**Business Rules:**

- Updating is allowed only if `status === "Pending"`.
- When updating:
  - Return the original stock (restore previous quantity).
  - Check stock availability for new quantity.
  - Subtract stock based on the updated quantity.
  - Recalculate `total_amount`.

---- END ---